Pimpri-Chinchwad Education Trust's
Pimpri-Chinchwad College of Engineering
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU)
ISO 21001:2018 Certified by TUV

Assignment No 3

Name: Soham Kulkarni
PRN: 124B1F056
Subject: Design and Analysis of Algorithms
Batch: A3
GitHub Link: https://github.com/SOHAM0007-CODER/DAA_ASSIGNMNET

1. Topic

A flood-relief boat has a limited carrying capacity W. Each supply item has a weight and utility value. Use Fractional Knapsack to select whole items or portions of divisible supplies so that total utility is maximized without exceeding the boat capacity.

2. Course Objective

- Understand the computational complexity of algorithms.
- Select a suitable algorithm design strategy for a practical problem.

3. Course Outcome

- Apply a Greedy strategy based on utility per unit weight.
- Compute fractional selections and analyze the conditions for greedy optimality.

Theory

Fractional Knapsack allows part of an item to be taken. Compute each item's profit-to-weight ratio, sort ratios from highest to lowest, and select supplies in that order. If the next item does not fully fit, take the fraction that fills the remaining capacity.

For positive weights and nonnegative utilities, choosing the highest available utility per kg is optimal when supplies are divisible. This differs from 0/1 Knapsack, where packages cannot be split. The supplied program treats all input items as divisible; indivisible medical kits require a different or mixed model.

Working of the Algorithm

- 1. Read item count, carrying capacity, and each item's weight and profit.
- 2. Compute profit/weight and sort items by decreasing ratio.
- 3. Take a whole item if it fits; otherwise take only the fraction that fits.
- 4. Print selected fractions, used weight, remaining capacity and maximum profit.

Complexity Analysis

Computing ratios takes O(N), sorting takes O(N log N), and the selection pass takes O(N). Therefore, the overall sorting-based method takes O(N log N) time.

Time Complexity

Best Case: O(N log N) with the sorting-based implementation.

Average Case: O(N log N).

Worst Case: O(N log N); sorting dominates the linear selection pass.

Space Complexity

O(N) total storage for the item records. Selection uses constant extra scalar variables; a typical comparison sort also uses O(log N) stack space.

Algorithm - Steps

- Step 1: Calculate ratios and arrange items from highest ratio to lowest.
- Step 2: Fill the boat with whole items while they fit.
- Step 3: Use a fraction for the final item if needed and add its proportional utility.

Pseudocode

```
sort items by decreasing profit/weight
remaining = capacity; totalProfit = 0
for each item in sorted order:
    if remaining == 0: break
    fraction = min(1, remaining / item.weight)
    totalProfit += fraction * item.profit
    remaining -= fraction * item.weight
print fractions, weight used and totalProfit
```

Flow of Execution

START -> Read supplies -> Calculate ratios -> Sort -> Select whole items or fractions -> Print result -> END.

6. Example

Capacity = 60 kg. Input (weight, profit) pairs are (20,60), (20,100), (5,30), (30,120), (15,40). Their ratio order is item 3, 2, 4, 1, 5. Take items 3, 2 and 4 fully, using 55 kg. Take 5/20 = 0.25 of item 1, adding 15 profit. Total weight = 60 kg and maximum profit = 30 + 100 + 120 + 15 = 265.

Conclusion

Fractional Knapsack maximizes utility by choosing the best profit per unit weight first. It is efficient and optimal for divisible supplies. The ability to take fractions is essential; the same greedy rule does not guarantee an optimum for indivisible packages.